SHRINAV WORKSHOP

Practice Assignment

API Fundamentals & Postman Mastery

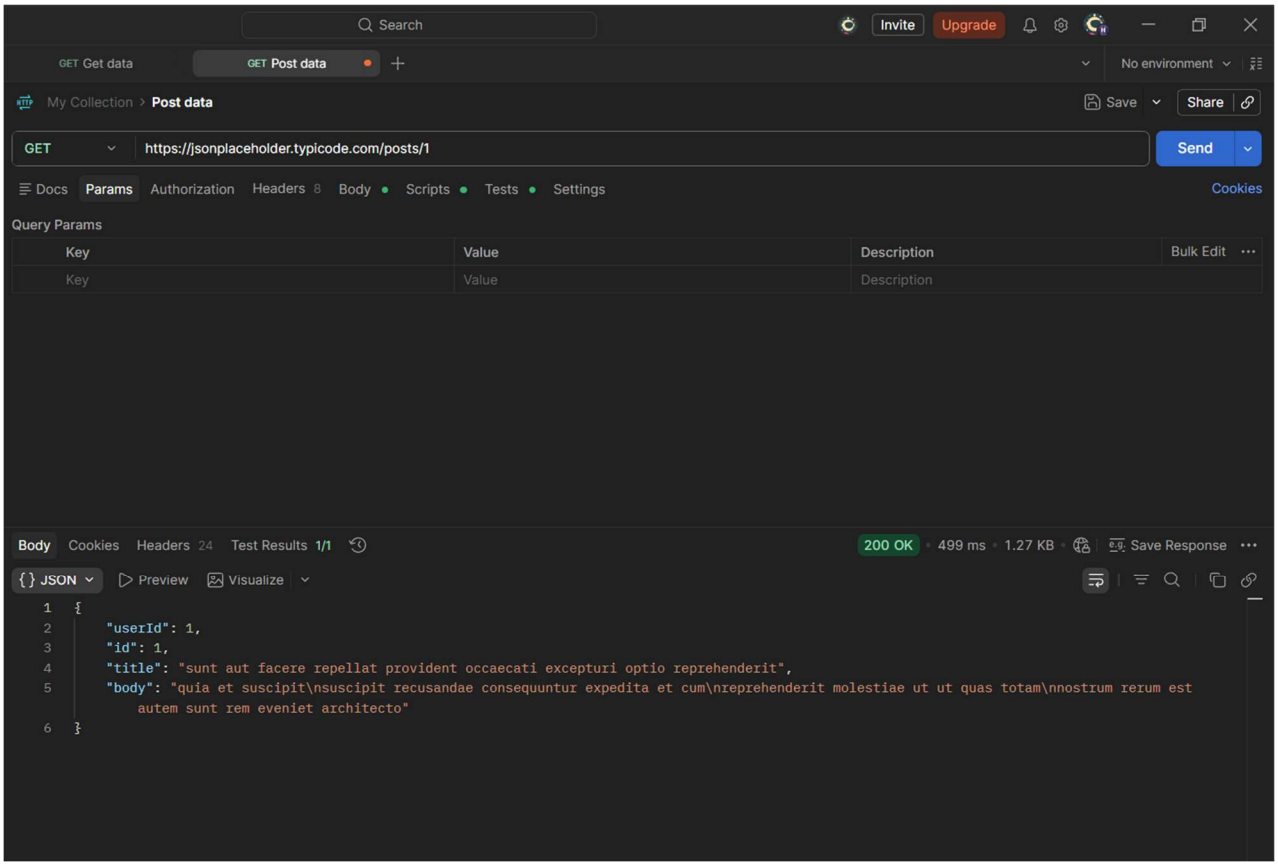
Complete every task in Postman using the JSONPlaceholder API. Read each instruction carefully — later tasks build on what you did in earlier ones.

Before you start: *you only need a browser and Postman. Base URL for every task: https://jsonplaceholder.typicode.com — no signup or API key required.*

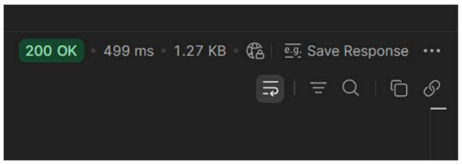
Tasks

TASK 1 Reading Data Without Headers

a) Send a GET request to /posts/1 — leave the Headers and Body tabs completely empty.



b) Record the status code you get back.

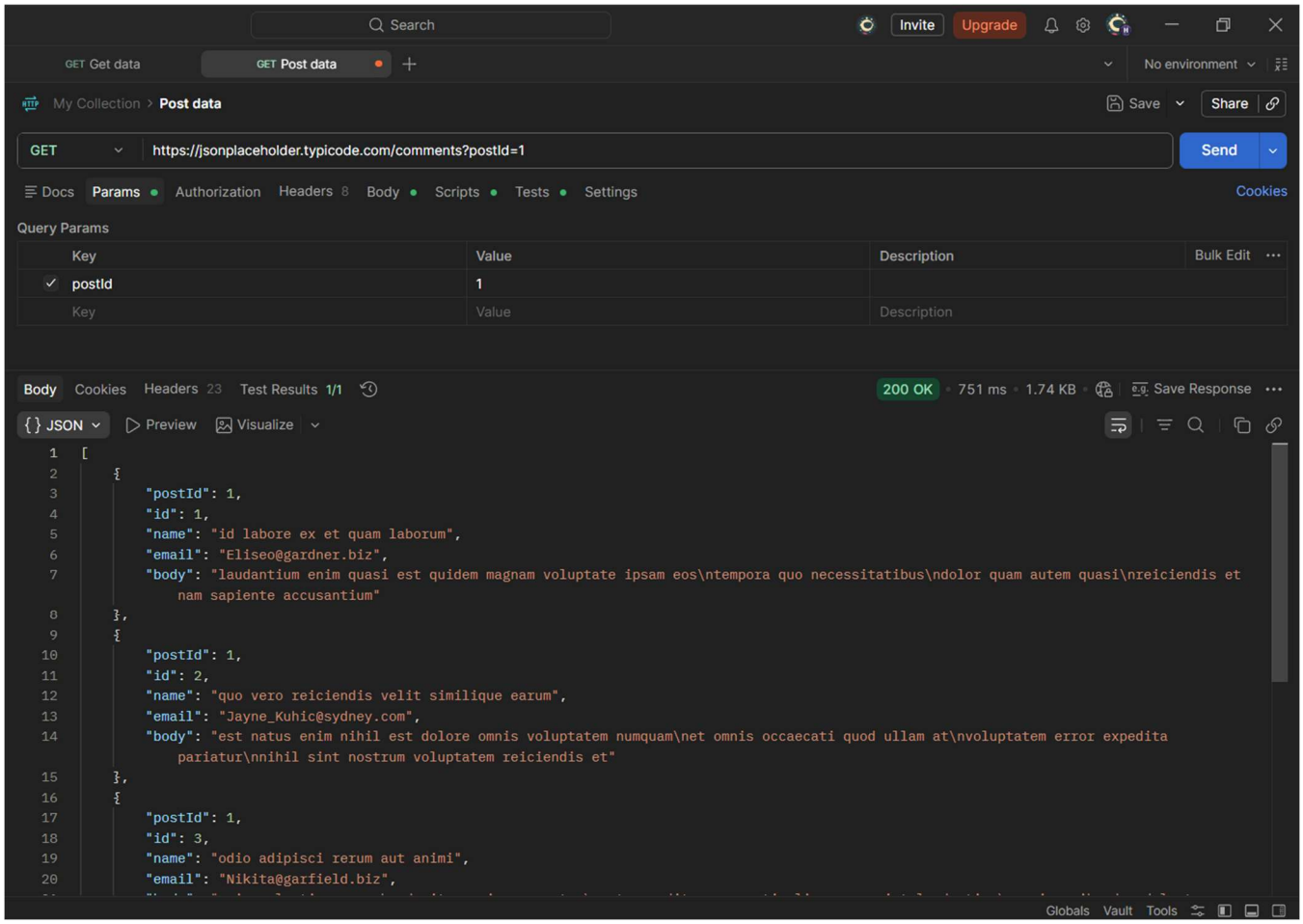


c) In one sentence, explain why this request works even with nothing in the Headers tab.

It works because this API endpoint doesn't require any special headers or authentication, so the request can be processed successfully without adding anything in the Headers tab.

TASK 2 Filtering With Query Parameters

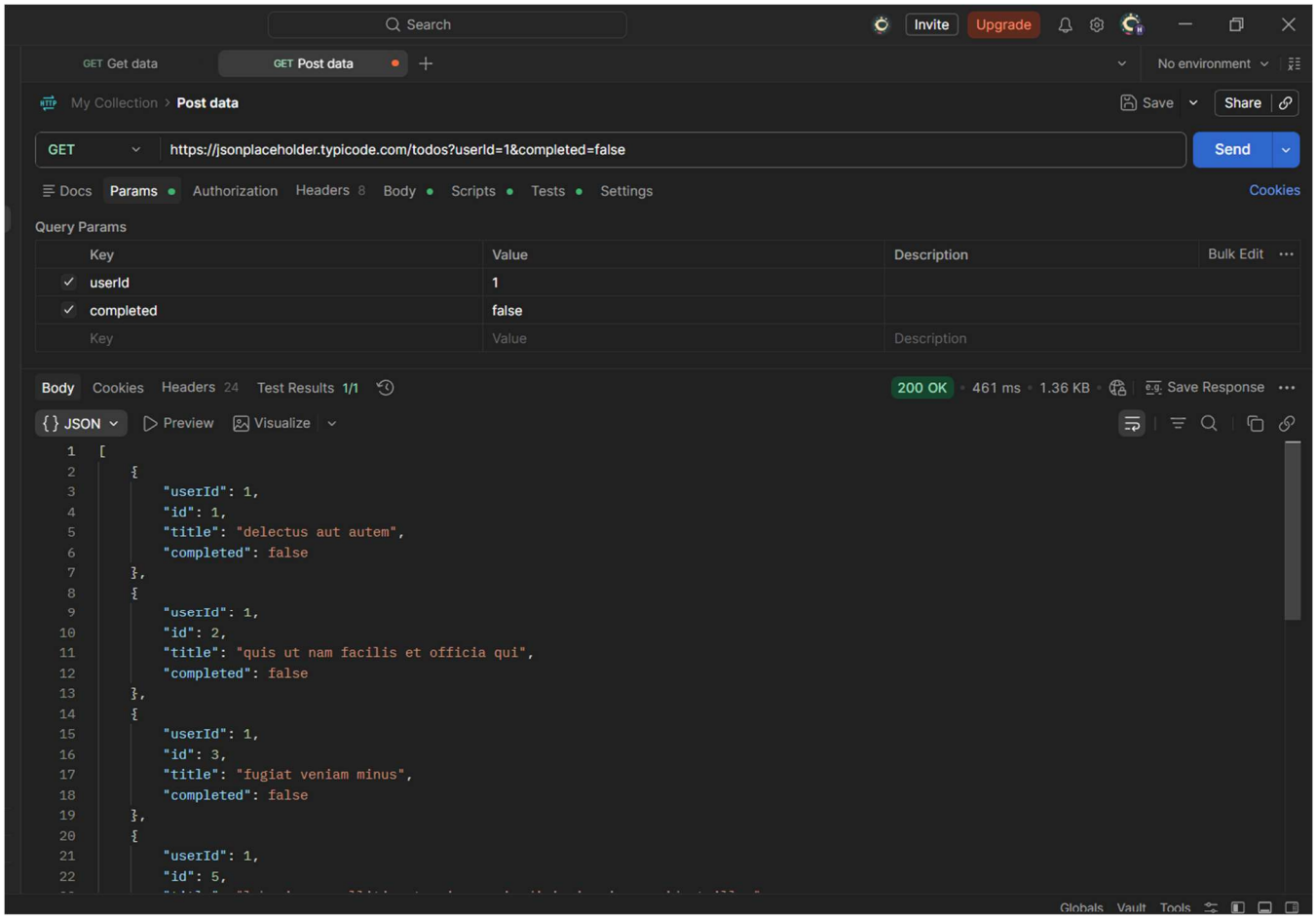
a) Send a GET request to /comments, and add a query parameter in the Params tab: postId = 1.



b) Confirm the URL automatically updates to include ?postId=1.

Yes the postman automatically updated the URL to include ?postID=1 when the query parameter and value was added.

c) Now combine two filters at once on /todos: userId = 1 and completed = false.



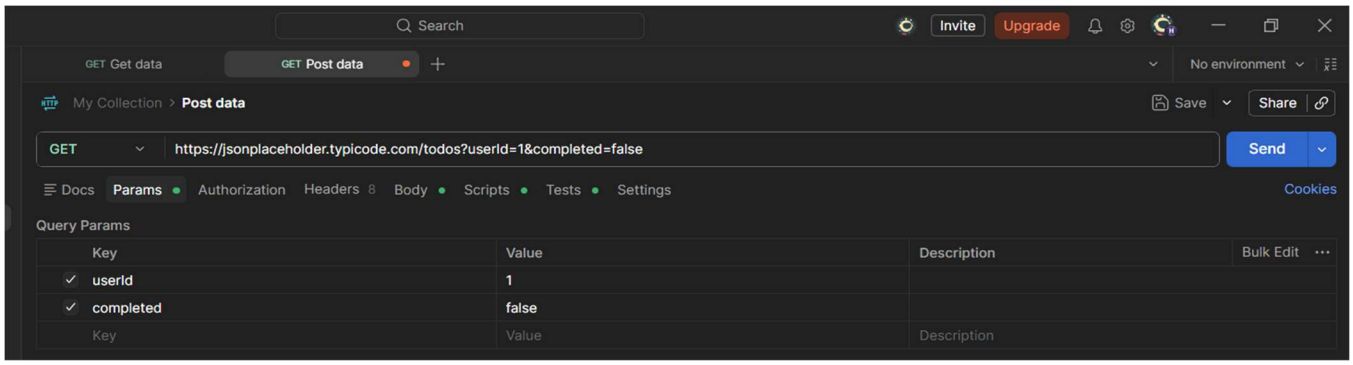
d) In one sentence, explain what changed in the results compared to not using any params.

Using the parameters returned a filtered response, so only the data matching the specified conditions was shown instead of the complete list.

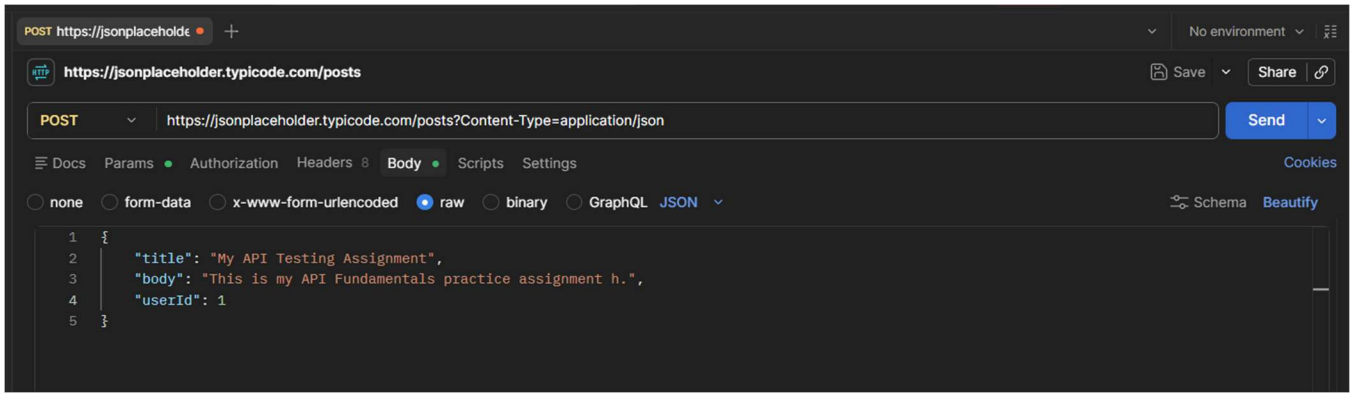
TASK 3 Sending Data With Headers & a Body

a) Send a POST request to /posts.

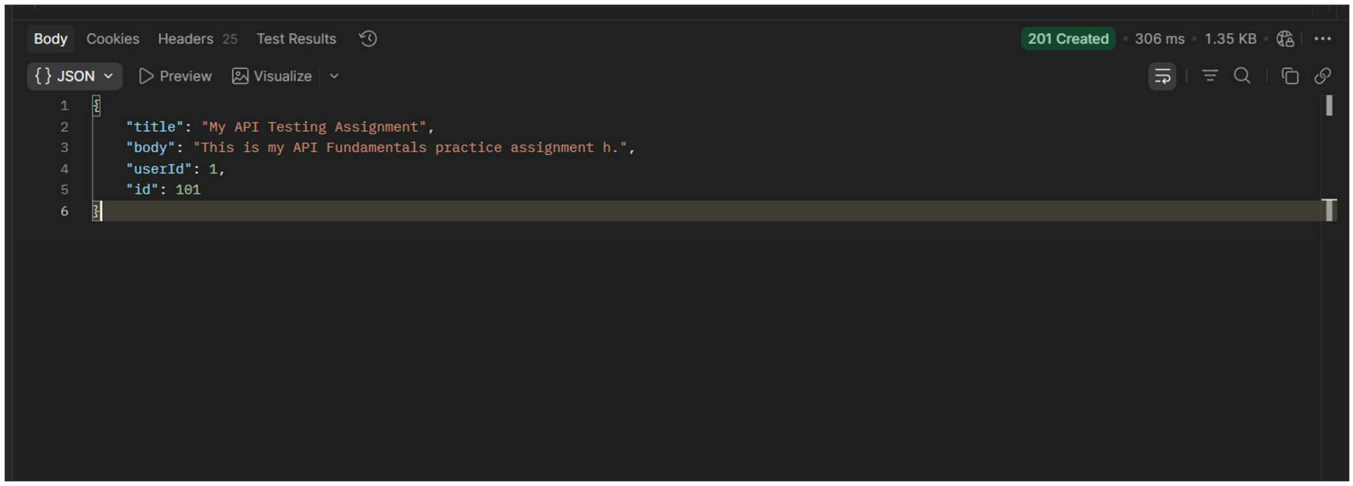
b) In the Headers tab, add: Content-Type: application/json



c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.

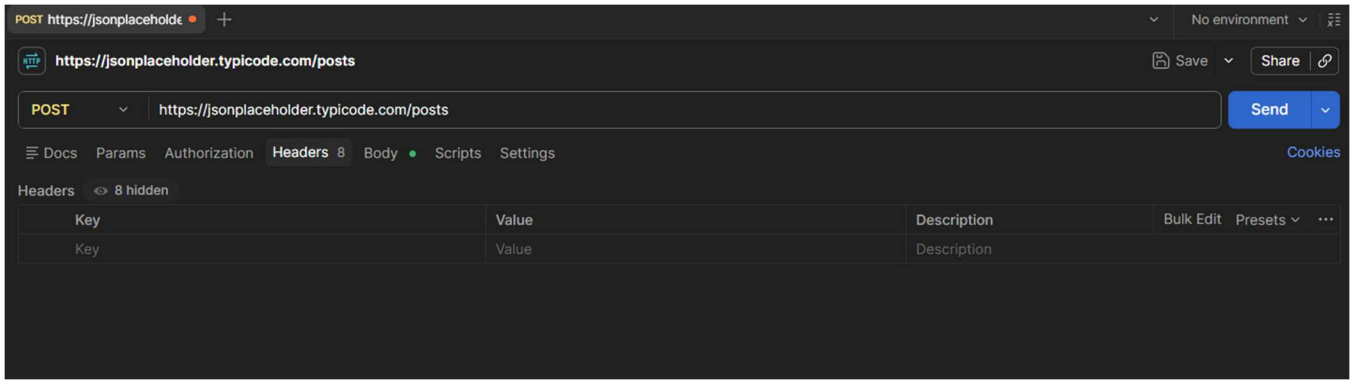


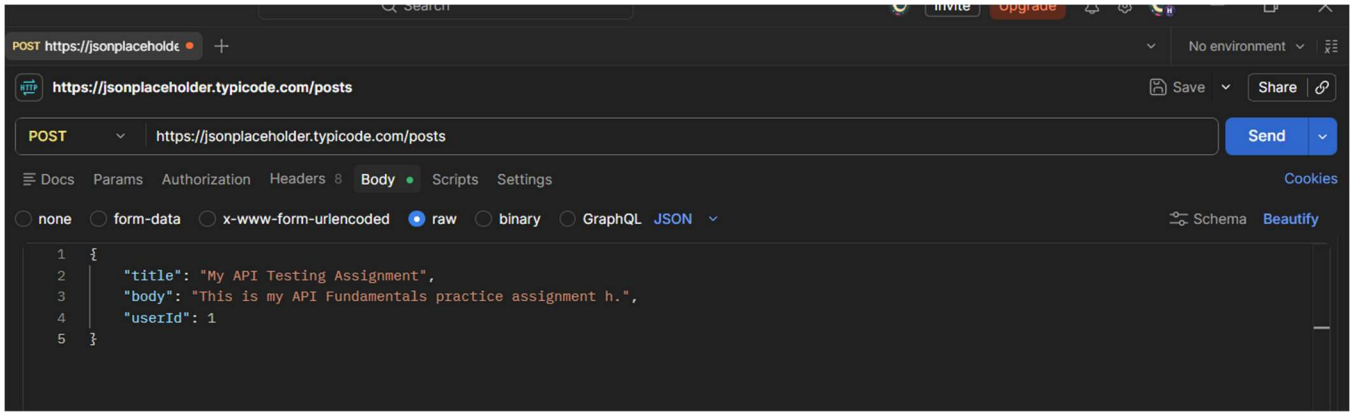
d) Record the status code and the new id the server assigned.



TASK 4 Headers vs. No Headers

a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.





b) Compare the response to Task 3 — is it different? Record what you observe.

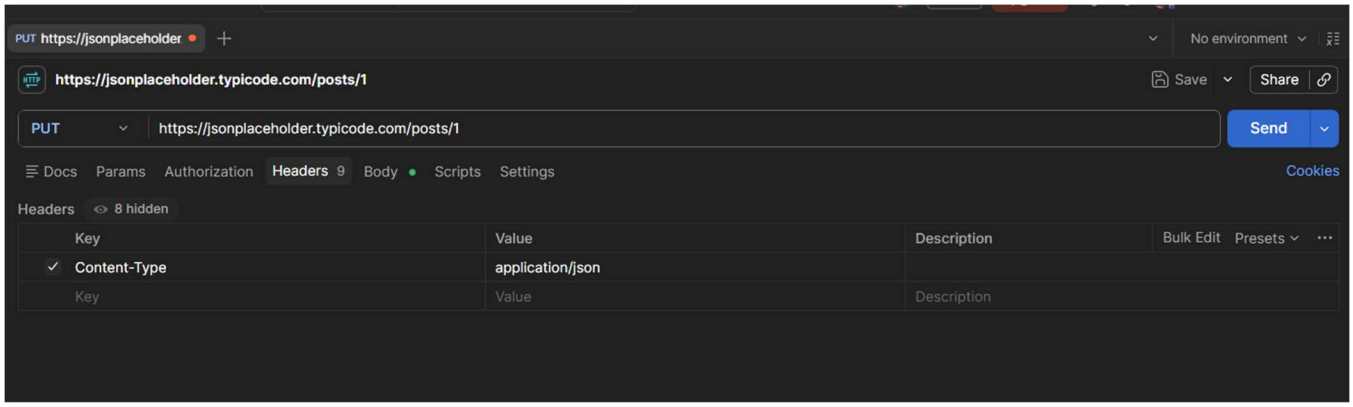
The response was almost the same as Task 3. JSONPlaceholder accepted the request even without the Content-Type header because it is a practice API and is designed to be flexible.

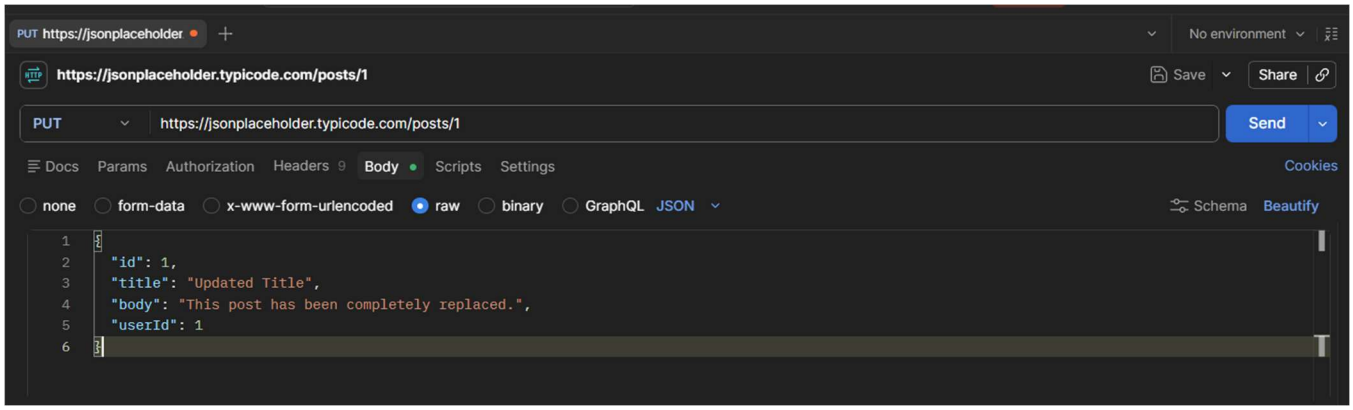
c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

Headers provide additional information about the request, such as the content type, authentication, or the response format the client expects. This practice API works without them because it is designed to be simple and public, but most real-world APIs require specific headers for the request to be processed correctly.

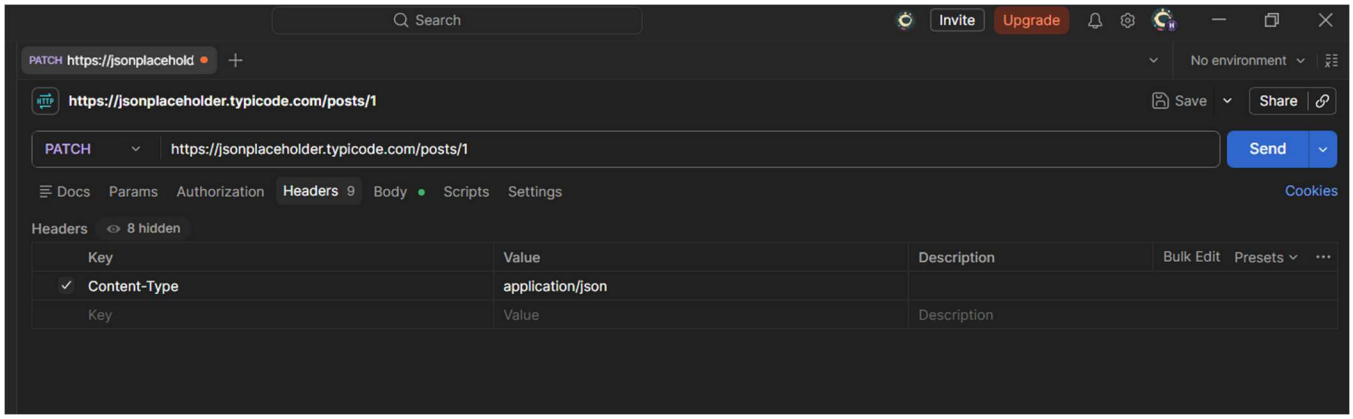
TASK 5 PUT vs. PATCH

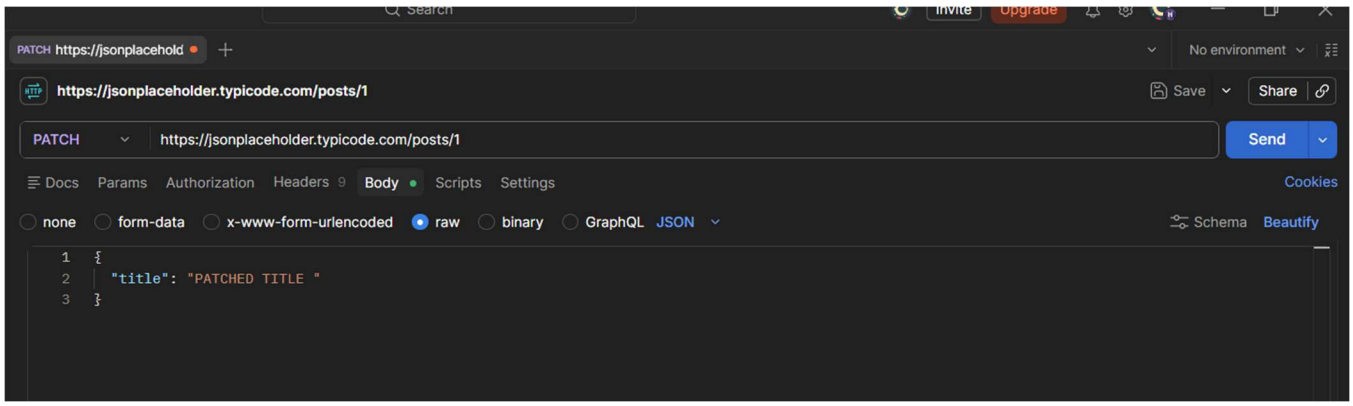
a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).





b) Send a PATCH request to the same /posts/1 that changes only the title.





c) In one sentence, explain the difference between what PUT sent and what PATCH sent.

PUT sent the entire resource and replaced all of its data, while **PATCH** sent only the specific fields that needed to be updated without replacing the whole resource.

Final Challenge — A Realistic Request Flow

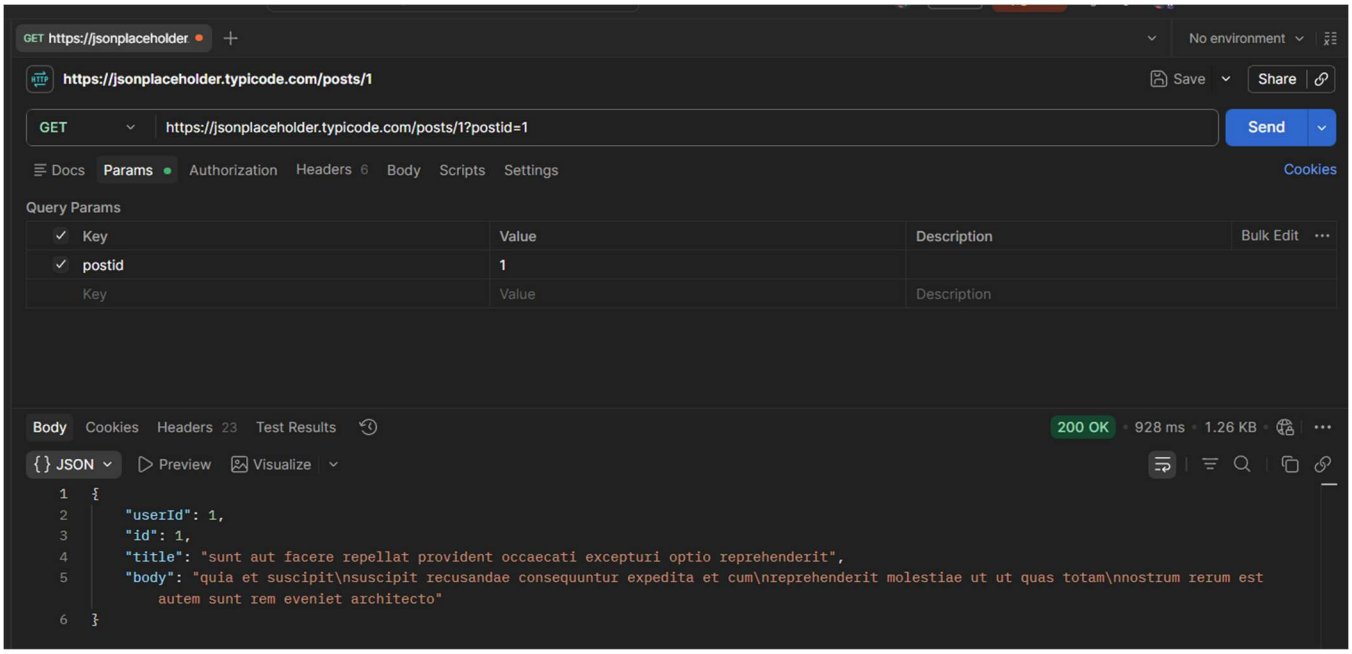
Concepts used: *this task combines query params, headers (with and without), and the request body — everything from Tasks 1 through 4 — in one flow.*

TASK 6 The Mini Feedback System

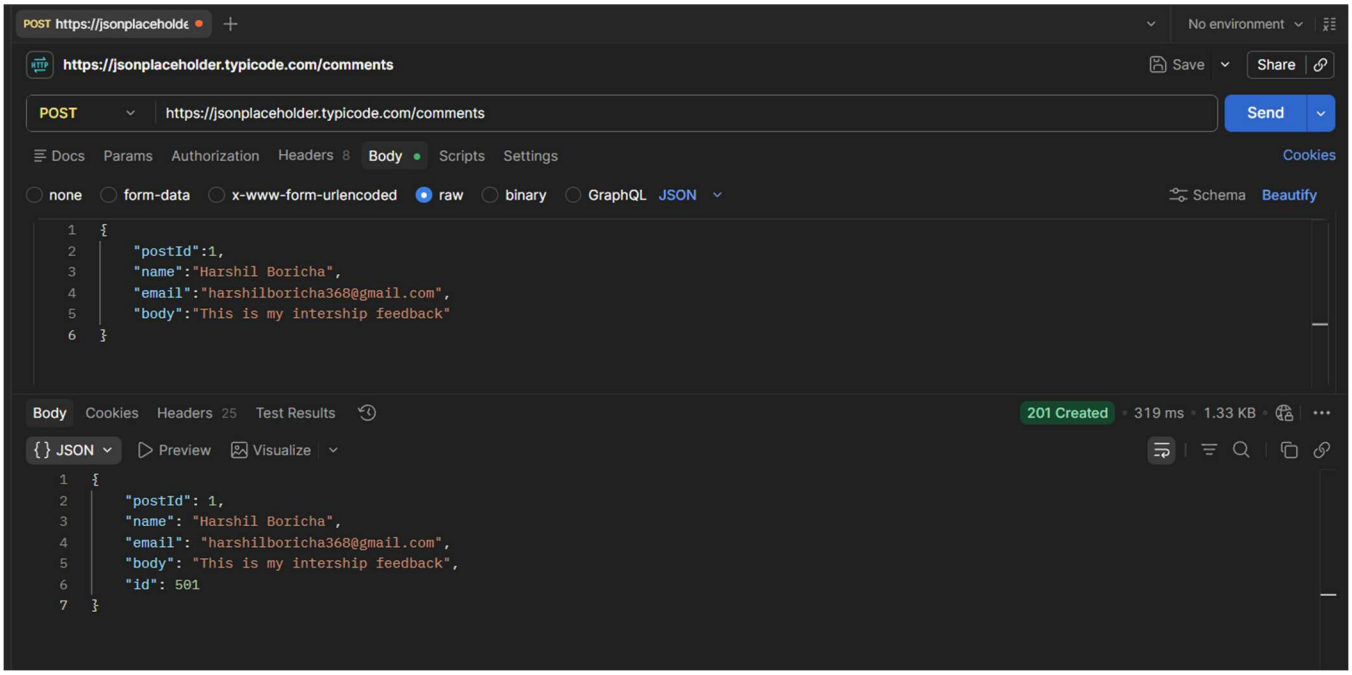
Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

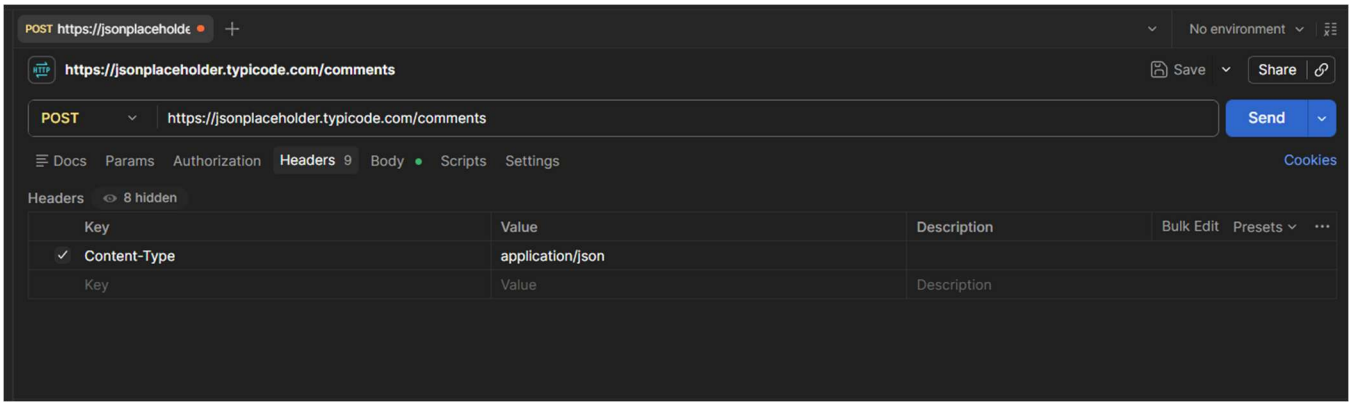
- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

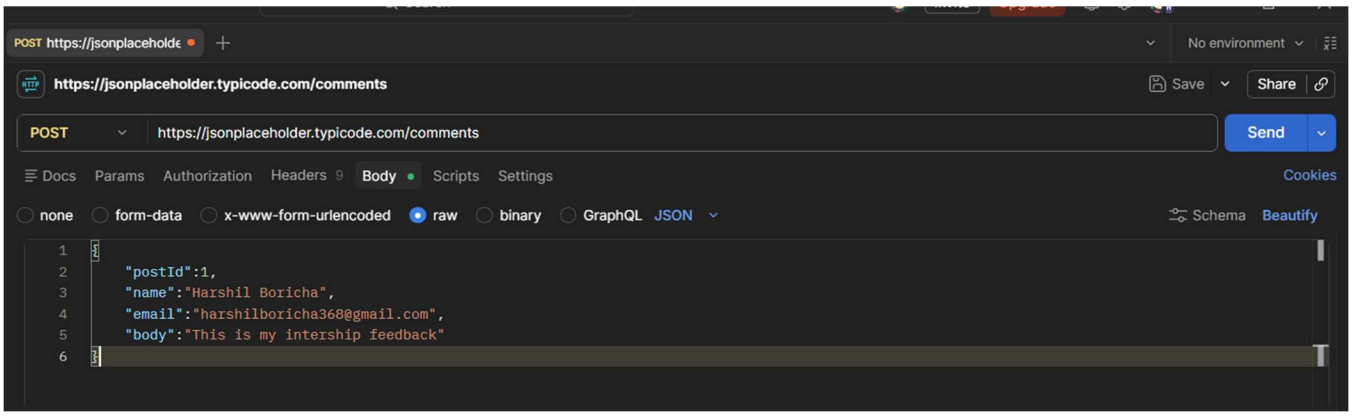


- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

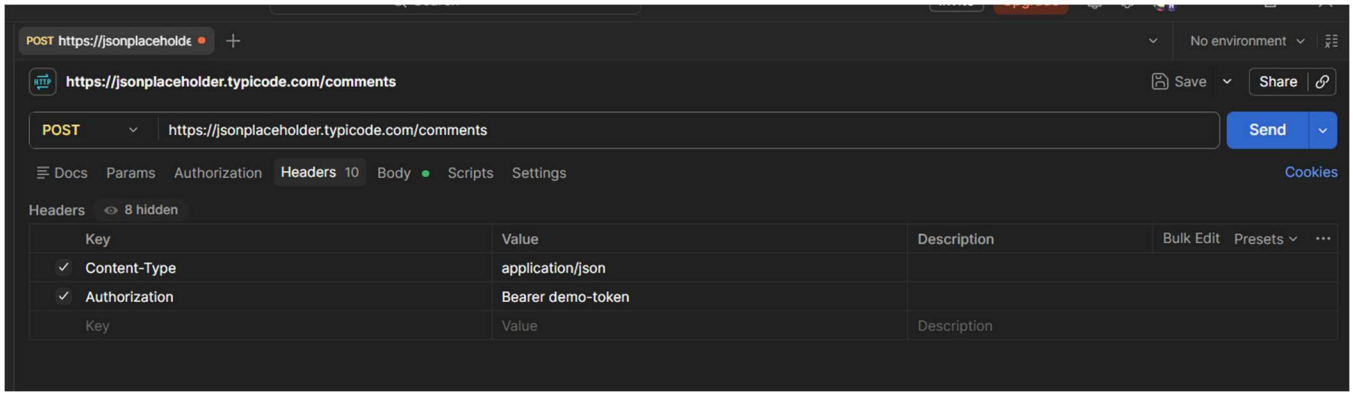


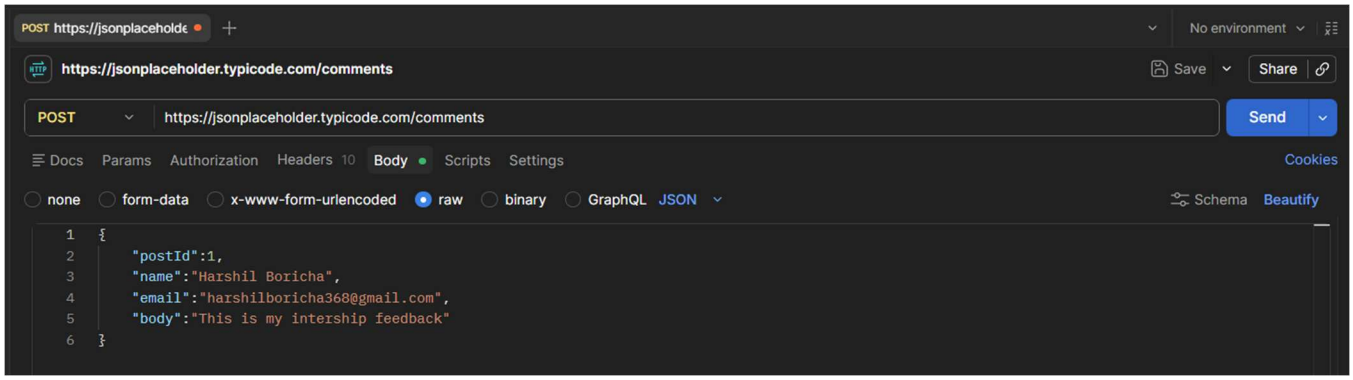
- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.





- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.





- e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
(a)	GET /comments?postId=1	**200 OK**
(b)	POST /comments (No Headers)	**201 Created**
(c)	POST /comments (Content-Type Header)	**201 Created**
(d)	POST /comments (Content-Type + Authorization Header)	**201 Created**

- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

In a real API, the server would check the **Authorization** header to verify the user's identity before allowing access to protected resources. If the authentication token was missing or invalid, the request would fail and the server would return an error such as **401 Unauthorized** or **403 Forbidden** instead of creating the resource. JSONPlaceholder ignores the Authorization header because it is a practice API and does not perform authentication, but real APIs use it to protect sensitive data and ensure that only authorized users can access or modify protected resources.

Submission Instructions

Submit your tasks inside your name folder in our GitHub repository. Make sure to include your requests and their response screenshots for each task.